



CS-695 Capstone Course

Conversational SQL Tutoring System

Presented By:

Shashwat Gautam

Problem & Motivation

- Many SQL tools and AI query generators act as black boxes — answers without reasoning
- Leads to shallow learning and over-reliance on automation
- Goal: teach SQL (joins, subqueries, aggregation, filtering) through dialogue and step-by-step explanations
- Focus on understanding, not just correct answers

Beneficiaries & Novelty

- Who benefits: Students in database/data science courses; professionals; educators (scalable feedback)
- What's new: Explanation-first, learner-centered design
- Adaptive explanations by proficiency level
- Mistake-aware feedback (why a query is wrong)
- Conceptual learning (how & why) over answer-only generation

System Architecture

- Frontend: Conversational UI, SQL editor, explanation panel
- Backend: Sandboxed execution (e.g. SQLite in-memory or containerized DB), APIs, session management
- AI layer: Intent detection, semantic parsing, NL-to-SQL, SQL analysis
- Explanation module: Step-by-step reasoning, common mistakes, alternatives — pretrained LLM + XAI principles

AI & NLP Components

- Intent detection: e.g. “write query” vs “explain my query”
- Semantic parsing and NL-to-SQL translation
- Hybrid: pretrained open-source LLMs + rule-based SQL validation (syntax, schema, safety)
- No large-scale custom training
 - prompting and optional adapter/LoRA for explanation style
- Rule-based checks detect errors and trigger targeted feedback

Explainability & Adaptation

- Explanations in natural language: what each clause does, why a join is needed, typical mistakes
- Adapt by user level: Beginners → simplified analogies; Advanced → concise technical reasoning
- Level inferred from dialogue or user setting

Dataset & Resources

- Spider text-to-SQL benchmark: 10k+ questions, 5.6k+ unique SQL, 200 DBs, 138 domains
- Optional: Spider 2.0 for more complex workflows
- Curated set of common misconceptions (e.g. GROUP BY, JOIN conditions) for mistake-aware feedback
- All public; no proprietary or sensitive data

Timeline

- Weeks 1–2: Literature & design → Proposal, architecture doc
- Weeks 3–4: Backend & sandbox → DB sandbox, API, safety
- Weeks 4–6: NL2SQL & analysis → Intent, NL-to-SQL, SQL checker
- Weeks 6–7: Explanation & UI → Explanation module, frontend
- Weeks 7–9: Integration & eval → End-to-end tests, user study prep
- Weeks 9–10: Report & demo → Final report, demo, presentation

Course & PLO Mapping

- Courses: Database systems, ML/NLP, Software engineering, AI/HCI
- PLO (MS CS): Analysis & design → architecture, requirements
- Implementation → full-stack system, AI integration
- Evaluation → user feedback, NL2SQL & explanation accuracy
- Communication → report, presentation; Ethics & safety → sandboxing, responsible AI

Challenges & Mitigation

- Explanation quality: LLM output may be vague/wrong → structured prompts, rule-based anchors, optional human review
- Schema complexity: Large schemas strain NL2SQL → Spider-sized schemas, limit context to relevant tables
- Scope creep: Strict MVP (SELECT-only, single sandbox, one LLM), phased features

Backup Plan

- If explanation/NL2SQL quality insufficient:
 - Template-based explanations keyed by error codes and query patterns
 - (B) Menu-driven problem selection + SQL submission only (reduce open-vocabulary)
 - (C) Prioritize SQL critique and fix suggestions over full NL-to-SQL
- Core contribution (explanation-first, mistake-aware tutoring) remains with reduced free-form generation

Expected Outcome

- Working prototype: Web-based conversational SQL tutor with editor, sandboxed execution, NL-to-SQL, feedback with explanations, basic proficiency adaptation
- Deliverables: (1) Deployable application, (2) Short user study or expert evaluation, (3) Final report and presentation
- Demonstrates applied AI, NLP, full-stack engineering at graduate capstone level; extensible base for multi-dialect SQL, richer student modeling

Questions & Discussion

Thank You!